

C++ vector operations and coding assessment prep ▾

How to find the max,min,their index , sort , in a vector in cpp. And also let me know all the concepts/techniques which are common in solving problems , as I have a 70min coding assessment on code signal platform.

Synthesized vector operations and competitive programming strategies for assessment preparation

C++ Vector Operations

cpp

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    vector<int> v = {3, 1, 4, 1, 5, 9, 2, 6};

    // Max & Min VALUE
    int maxVal = *max_element(v.begin(), v.end());
    int minVal = *min_element(v.begin(), v.end());

    // Max & Min INDEX
    int maxIdx = max_element(v.begin(), v.end()) - v.begin();
    int minIdx = min_element(v.begin(), v.end()) - v.begin();

    // Sort ascending
    sort(v.begin(), v.end());

    // Sort descending
    sort(v.begin(), v.end(), greater<int>());

    // Custom sort (by absolute value)
    sort(v.begin(), v.end(), [](int a, int b) {
        return abs(a) < abs(b);
    });

    // Binary search (only on sorted vector)
    bool found = binary_search(v.begin(), v.end(), 5);

    // Lower bound = first element >= target
    auto it = lower_bound(v.begin(), v.end(), 5);
    int idx = it - v.begin();

    // Upper bound = first element > target
    auto it2 = upper_bound(v.begin(), v.end(), 5);
}
```

Common Techniques for CodeSignal

1. Two Pointers

cpp



```
// Check if pair with target sum exists in sorted array
int l = 0, r = v.size() - 1;
while (l < r) {
    int sum = v[l] + v[r];
    if (sum == target) { /* found */ break; }
    else if (sum < target) l++;
    else r--;
}
```

2. Sliding Window

```
cpp

// Max sum subarray of size k
int windowSum = 0, maxSum = 0;
for (int i = 0; i < k; i++) windowSum += v[i];
maxSum = windowSum;
for (int i = k; i < v.size(); i++) {
    windowSum += v[i] - v[i - k];
    maxSum = max(maxSum, windowSum);
}
```

3. Prefix Sum

```
cpp

vector<int> prefix(v.size() + 1, 0);
for (int i = 0; i < v.size(); i++)
    prefix[i + 1] = prefix[i] + v[i];

// Sum from index l to r (inclusive) in O(1)
int rangeSum = prefix[r + 1] - prefix[l];
```

4. HashMap / Frequency Count

```
cpp

unordered_map<int, int> freq;
for (int x : v) freq[x]++;

// Check if element exists
if (freq.count(5)) { /* exists */ }
```

5. HashSet (for duplicates / lookup)

```
cpp

unordered_set<int> seen;
for (int x : v) {
    if (seen.count(x)) { /* duplicate found */ }
    seen.insert(x);
}
```

6. Stack (parentheses, next greater element)

```
cpp

stack<int> st;
for (int x : v) {
    while (!st.empty() && st.top() < x) st.pop();
    st.push(x);
}
```

7. Queue / BFS

```
cpp

queue<int> q;
q.push(start);
while (!q.empty()) {
    int node = q.front(); q.pop();
    // process node, push neighbors
}
```

8. Priority Queue (Heap)

```
cpp

priority_queue<int> maxHeap;           // max on top
priority_queue<int, vector<int>, greater<int>> minHeap; // min on top
maxHeap.push(5);
int top = maxHeap.top(); maxHeap.pop();
```

9. String Tricks

```
cpp
```

```
// Split string by space
stringstream ss(str);
string word;
while (ss >> word) words.push_back(word);

// Reverse a string
reverse(str.begin(), str.end());

// To uppercase / lowercase
transform(str.begin(), str.end(), str.begin(), ::tolower);
```

10. GCD / LCM

```
cpp

int g = __gcd(a, b);
int l = (a / g) * b; // avoid overflow
```

CodeSignal-Specific Tips

Tip	Why
Use <code>bits/stdc++.h</code>	Includes everything
Prefer <code>unordered_map</code> over <code>map</code>	$O(1)$ vs $O(\log n)$
Watch for <code>int</code> overflow → use <code>long long</code>	Very common gotcha
Read constraints first	Tells you expected time complexity
$O(n^2)$ is fine up to $\sim 10^4$, $O(n \log n)$ up to $\sim 10^6$	Helps pick algorithm fast
Edge cases: empty input, single element, all same values	Test these mentally

The most frequently tested patterns on CodeSignal are **sliding window**, **hashmaps for frequency/lookup**, and **two pointers** — make sure those are solid.

📄 🔊 🗣️ 🔄

```
unordered_map<int, int> freq;
for (int x : v) freq[x]++;
```

```
// Check if element exists
if (freq.count(5)) { /* exists */ }
```

Does this add x as a key in the empty req map

Thought for 3s

Yes. In C++, `operator[]` on an `unordered_map` **auto-inserts** the key with a default value if it doesn't exist.

So `freq[x]++` does this internally:

1. If `x` **doesn't exist** → inserts `{x, 0}` (int defaults to 0), then increments → `{x, 1}`
2. If `x` **exists** → just increments the existing value

This is why `freq.count(key)` is safer than `freq[key]` for just checking existence — using `[]` to check would silently insert the key with value `0`.

cpp

```
// BAD - inserts key=5 with value 0 if not present
if (freq[5]) { }
```

```
// GOOD - just checks, no insertion
if (freq.count(5)) { }
```

```
// Also fine (C++20)
if (freq.contains(5)) { }
```

📄 🔊 🗨️ 🔄 🔄

